

WEB TEKNOLOJİLERİ ve PROGRAMLAMA

2. Ders: HTML, CSS, Javascript Temelleri



SÜLEYMAN
DEMİREL
ÜNİVERSİTESİ



BİLGİSAYAR
MÜHENDİSLİĞİ

HTML Temelleri



- HTML bir programlama dili değildir. Bir markup (işaretleme) dilidir. (Hyper Text Markup Language).
- Web sayfaları oluşturmak için tanımlanmış bir standarttır.
- Tarayıcıya içeriğin nasıl gösterileceğini söyler.
- 2 gruba ayırabiliriz:
 - Bir açma bir de kapama tag'inden oluşan container tag'leri: `<b>Kalın Yazı</b>`
 - Tek başına kullanılan tag'ler: `<hr>` yatay çizgi ekler.
- Tüm HTML dokümanları doküman tipini belirten tag/etiket ile başlamalıdır. Bu etiket en üstte yer almalıdır: **`<!DOCTYPE html>`**
- HTML dokümanının kendisi `<html>` etiketi ile başlar `</html>` etiketi ile biter.
- HTML dokümanının görünür tarafı **`<body>`** **`</body>`** etiketinin içindedir.
- Bir etiket başka bir etiketin içinde yer alabilir.



HTML Temelleri



- Başlıklar 1'den (<h1>) 6'ya kadar (<h6>) tanımlanmıştır.
 - Örnek: <h1> Başlık 1 </h1>
- Paragraflar <p> etiketi ile tanımlanır.
 - <p> Bu bir paragraf </p>
- Linkler <a> etiketi ile tanımlanır.
 - SDÜ ana sayfası
 - **href** gidilecek bağlantıyı tanımlar.
- Resimler etiketi ile tanımlanır.
 -
 - **src** resimin kaynağını **alt** üzerine geldiğinde gösterilecek metni tanımlar.
- Alt satıra
 etiketi ile geçebilirsiniz.
- Tüm elementlerin kendine has özellikleri (attribute) vardır.
 - a etiketinin href özelliği
 - img etiketinin src özelliği



HTML Temelleri



- **style** özelliği elementlere renk, font, büyüklük özellikleri kazandırır.
 - `<p style="color:red;">Bu bir paragraftır.</p>`
 - `özellikadı="özellik:değer;"`
 - `<body style="background-color:blue;">`
- Özelliklerin aldıkları değerleri mutlaka tırnak içinde yazın. Genelde çift tırnak kullanılır.
- HTML'de çok çeşitli formatlama elementleri vardır.
 - `<b>` Kalın metin
 - `<strong>` Önemli metin
 - `<i>` italik metin
 - `<small>` küçük metin
 - `<sup>` üst metin



HTML Temelleri



- Renkleri renk adlarıyla gösterebilirsiniz.
 - `<h1 style="background-color:Tomato;">Tomato</h1>`
- Arkaplan rengi ya da metin rengi belirleyebilirsiniz.
 - Örnek-1
 - Örnek-2
- Renkler rgb, hsl ya da hex formatında da belirtilebilir.
 - Örnek
- HTML dokümanınıza CSS (Cascading Style Sheets, Basamaklı Stil Sayfaları) ile stil verebilirsiniz.
 - İçerden eklemeli
 - Dış kaynaktan eklemeli



HTML Temelleri



- Link verirken linklerin nasıl açılacağını belirtebilirsiniz.
 - `_self`: aynı sayfada açılır.
 - `blank`: yeni bir sayfada açılır.
 - `_parent`: Bir parent çerçevesi varsa onun içinde açılır.
- Bir e-posta adresine link verebilirsiniz.
 - `<a href="mailto:asimyuksel@sdu.edu.tr">E-Posta Gönder</a>`
- HTML dokümanınızın arkasını kaplayan resimler ekleyebilirsiniz.
 - `<p style="background-image: url('resim.jpg');">`
- Verilerinizi `table`, `tr`, `th`, `td`, etiketlerini kullanarak gösterebilirsiniz.
 - Örnek
- Verilerinizi sıralı ya da sırasız olarak **ul** ve **li** etiketleriyle liste halinde gösterebilirsiniz.
 - Sırasız
 - Sıralı



HTML Temelleri



- Her elementin varsayılan bir gösterim değeri vardır. Bu değerler iki grupta yer alır: **block** ve **inline**
- Block düzeyindeki elementler her zaman yeni bir satırdan başlar. Tarayıcılar otomatik olarak sağına ve soluna boşluk atarlar.
- Block düzeyindeki elementler varolan tüm genişliği kullanırlar.
 - Bu elementlere örnek: <p>, <div>
- Block düzeyindeki tüm elementler: <address>, <article>, <aside>, <blockquote>, <canvas>, <dd>, <div>, <dl>, <dt>, <fieldset>, <figcaption>, <figure>, <footer>, <form>, <h1>-<h6>, <header>, <hr>, , <main>, <nav>, <noscript>, , <p>, <pre>, <section>, <table>, <tfoot>, , <video>
- Inline elementler yeni bir satırda başlamaz.
 - Örnek: Hello World
- Inline elementler: <a>, <abbr>, <acronym>, , <bdo>, <big>,
, <button>, <cite>, <code>, <dfn>, , <i>, , <input>, <kbd>, <label>, <map>, <object>, <output>, <q>, <samp>, <script>, <select>, <small>, , , <sub>, <sup>, <textarea>, <time>, <tt>, <var>



HTML Temelleri



- HTML **class** özelliği bir elemente sınıf atamayı sağlar. Bu sayede benzer bilgiler içeren elementleri gruplayarak, aynı fontta, renkte ya da büyüklükte olmalarını sağlayabilirsiniz.
- Farklı elementler aynı class'a sahip olabilir. Farklı farklı sınıflar da tanımlayabilirsiniz.
 - Örnek
- Elementlerinize benzersiz id'ler verebilirsiniz. Bu sayede Javascript aracılığı ile istediğiniz elemente ulaşarak o elementle ilgili işlemler gerçekleştirebilirsiniz.
 - Örnek-1
 - Örnek-2
- `<script>` etiketiyle dokümanınıza Javascript ekleyebilirsiniz.
 - Örnek
- Dosya yolları:
 - `` : resim.jpg html dosyasıyla aynı yerdedir.
 - `` : resim.jpg html dosyasıyla aynı yerde bulunan images klasöründedir.
 - `` : resim.jpg html dosyasının bulunduğu kök dizindeki images klasöründedir.
 - `` : resim.jpg html dosyasının bulunduğu dizinin bir üst dizinindedir.



HTML Temelleri



- <head> elementi <title>, <style>, <meta>, <link>, <script>, and <base> elementleri için container/kapsayıcı elementtir.
- <title> elementi sayfanızın başlığını belirler.
- <link> elementi ile stil dosyalarınızı kullanabilirsiniz.
 - <link rel="stylesheet" href="stildosyam.css">
- <meta> elementi karakter setini, sayfa tanımlarını, anahtar kelimeleri, doküman yazarını, viewport ayarlarını/responsive belirtmek için kullanılır.
 - Örnek
- <section> elementi bir dokümanda bölüm tanımlar. Başlığı olan bir çeşit gruplama elementidir.
 - Örnek
- <article> bağımsız, kendi kendine yeten, kendi içeriği olan bir elementtir. Forum postu, blog postu gibi.
 - Örnek
- Section elementi article elementi içinde, article elementi de section elementi içinde tanımlanabilir.
- <footer> elementi alt bilgi (copyright gibi) göstermeyi sağlar.



HTML Temelleri



- `<nav>` elementi navigasyon linkleri tanımlamayı sağlar. Örnek
- `<aside>` elementi içerik dışında ek bilgi sunmak için kullanılır. Örnek
- `<figure>` elementi kendi kendine yeten, kendi içeriği olan içerikleri tanımlar. Diyagramlar, fotoğraflar, kod listeleri gibi. Örnek
- Emojiler UTF-8 karakter setindeki özel karakterlerdir. Emojiler resim ya da ikon gibi görünürler ama değildir.
- Çoğu UTF-8 karakteri klavyeden yazılamaz. Ancak sayısal değerlerle ifade edilebilir.
 - A:65 B:66
- Tarayıcının girdiğiniz karakteri anlaması için **&#** ile başlamalı ; ile bitirmelisiniz.
 - 😊 **😄**
 - 🥰 **😍**
 - 💕 **💗**
- HTML sayfalarının doğru bir şekilde gösterilmesi için kullanılan karakter setinin belirtilmesi gerekir.
 - `<meta charset="UTF-8">`



HTML Temelleri



- Bazı özel karakterler

Sonuç	Açıklama	Özel Karakter	Sayısal Karşılık
	non-breaking space	 	
<	less than	<	<
>	greater than	>	>
&	ampersand	&	&
"	double quotation mark	"	"
'	single quotation mark (apostrophe)	'	'
¢	cent	¢	¢
£	pound	£	£
¥	yen	¥	¥
€	euro	€	€
©	copyright	©	©
®	registered trademark	®	®



HTML Temelleri



- HTML formları kullanıcıdan bilgi toplamak için kullanılan elementlerdir. Formlardan alınan bilgiler genelde işlenmek üzere sunucu tarafına gönderilir. Form etiketi ile belirtilir.
- Çok farklı giriş elementleri için kapsayıcı görevi görür.
 - `<form> ..... Form elementleri..... </form>`
- `input` elementi en sık kullanılan elementtir.
 - `<input type="text">` : tek satırlı metin girdisi. Size özelliği ile genişliğini artırabilirsiniz.
 - `<input type="radio">` : Bir çok seçenekten birini seçmek için radyo düğmesi
 - `<input type="checkbox">` : Birden fazla seçeneği seçmek için onay kutusu
 - `<input type="submit">` : Formu göndermek için gönder tuşu
 - `<input type="button">` : Standart tıklanabilir tuş
- **action** özelliği form gönderildiğinde ne yapılması gerektiğini belirtir.
 - Örnek
- **target** özelliği form gönderildikten sonra gelen verinin nerede gösterileceğini belirtir. Yeni bir sayfada mı, aynı sayfada mı?
 - Örnek



HTML Temelleri



- **method** özelliği form gönderilirken kullanılacak http metodunu (GET, POST gibi) belirler. Örnek
- GET ve POST farkları
 - GET form verisini URL'ye ekler, POST eklemez.
 - Hassas içerikler asla GET ile gönderilmemeli.
 - GET için URL'de bir limit vardır. POST için bir limit yoktur.
 - POST büyük boyutlu veriler göndermek için kullanılır.
 - POST veriyi HTTP isteğinin gövdesine ekler.
- **select** elementi aşağı açılır liste tanımlar. Liste elemanları **option** elementi ile belirlenir. Önceden seçilmiş bir liste elemanı tanımlamak için **selected** özelliği kullanılır. Örnek
- **textarea** elementi birden fazla satır içeren metin girdisine izin verir. Örnek
- **button** elementi tıklanabilir bir düğme tanımlar. Örnek
- **fieldset** elementi formdaki verileri gruplamak için kullanılır. Örnek
- **<input type="password">** şifre alanı tanımlar. Örnek
- **<input type="reset">** formdaki tüm değerleri varsayılan değerlerine resetleyen bir düğme tanımlar. Örnek



HTML Temelleri



- **<input type="color">** renk seçmenizi sağlar. Örnek
- **<input type="date">** tarih seçmenizi sağlar. Örnek
- **<input type="email">** eposta formatında eposta adresi girmenizi sağlar. Örnek
- **<input type="image">** resim biçiminde bir gönder tuşu tanımlar. Örnek
- **<input type="file">** dosya seçmenizi sağlayan bir düğme tanımlar. Örnek
- **<input type="hidden">** Gizli bir girdi alanı tanımlar. Genelde form gönderildiğinde kullanıcılar tarafından görünmeyecek ve değiştirilemeyecek bir veri eklemek için kullanılır. Ancak tarayıcılardaki incele düğmesiyle kaynaktan görünebilir. Güvenlik amaçlı asla kullanmayın. Örnek
- **<input type="number">** Sayısal bir değer girmenizi sağlar. Örnek
- **<input type="tel">** Formatlı olarak telefon numarası girmenizi sağlar. Örnek
- **value** özelliği input elementine ilk değer atamanızı sağlar. Örnek
- **readonly** özelliği input elementinin sadece okunabilir olacağını belirtir. Örnek
- **pattern** özelliği girişi sınırlandıracak regex yazmanızı sağlar. Örnek
- **required** ilgili alanın girilmesinin zorunlu olmasını sağlar.
- **autofocus** sayfa yüklendiğinde ilgili alana otomatik olarak odaklanır. Örnek



HTML Temelleri

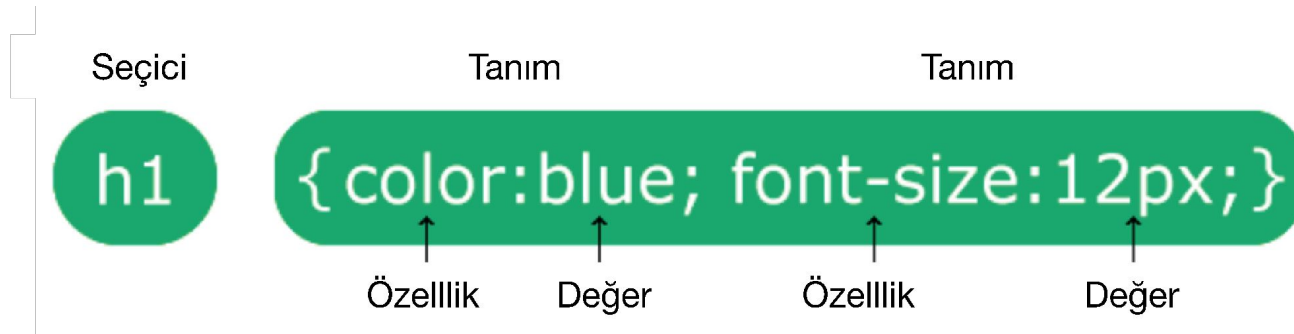


- **video** elementi web sayfasında video göstermenizi sağlar. Örnek
- **audio** elementi web sayfasında ses çalmanızı sağlar. Örnek
- **object** elementi html dokümanına nesne eklemek için kullanılır. Applet, PDF okuyucu, player, başka bir html dosyasını nesne olarak ekleyebilirsiniz. Örnek
- HTML dosyasına youtube videosu da ekleyebilirsiniz. Bunun için iframe elementini kullanabilirsiniz. Örnek
- HTML Geolocation API'si ile kullanıcının konumunu alabilirsiniz. Örnek
- Sürükle bırak özelliğini içeren sayfalar oluşturabilirsiniz. Örnek
- Tarayıcıda tutmanız gereken verileri Web Storage özelliği ile tutabilirsiniz. Cookie'lere daha iyi bir alternatif sunar. Daha büyük boyutlu veriler saklayabilirsiniz.
 - localStorage nesnesi ile veri tutma
 - sessionStorage ile oturum bilgisi tutabilirsiniz.
- Web Worker API ile arka planda çalışan işler tanımlayabilirsiniz. Örnek
- Server-Sent Events (SSE) API ile sunucudan otomatik güncellemeler alabilirsiniz. Örnek



CSS Temelleri

- CSS: Cascading Style Sheets, Basamaklı Stil Sayfaları
- HTML sayfalarına stil vermek (tasarım, yerleşim gibi), onların nasıl gösterileceğini tanımlamak için kullanılan bir dildir.
- CSS kuralı bir selector/seçici ve tanım bloğundan (declaration block) oluşur.



- Seçici hangi HTML elementine stil vereceğinizi gösterir.
- Tanım bloğu bir ya da daha fazla tanım içerir ve noktalı virgül ile ayrılır. Tüm blok süslü parantez içine yazılır. Örnek

CSS Temelleri



- Seçiciler 5 kategoride yer alır.
 - **Basit seçiciler:** Elementleri isim, id, class'a göre seçerler. Örnek-1, Örnek-2, Örnek-3
 - **Birleştirici/Combinator seçiciler:** Elementlerin birbirleri arasındaki belirli ilişkilerine göre seçerler. Örnek-1, Örnek-2, Örnek-3, Örnek-4
 - **Pseudo-class seçiciler:** Elementler belli bir duruma göre (Fare üzerine geldiğinde gibi) seçilir. Örnek-1, Örnek-2, Örnek-3, Örnek-4, Örnek-5, Örnek-6
 - **Pseudo-elements seçiciler:** Bir elementin belli bir bölümünü (ilk harf, ilk satır gibi) seçerler ve stil verirler. Örnek-1, Örnek-2, Örnek-3, Örnek-4, Örnek-5, Örnek-6, Örnek-7, Örnek-8
 - **Özellik/Attribute seçiciler:** Elementleri belli özelliklere ya da özellik değerlerine göre seçerler. Örnek-1, Örnek-2, Örnek-3, Örnek-4, Örnek-5, Örnek-6



CSS Temelleri



- Tüm elementleri etkileyen seçicilere evrensel/universal seçiciler denir. Örnek
- Aynı stil tanımlarına sahip olacak elementleri grup seçici ile birleştirebilirsiniz. Örnek
- CSS'leri 3 farklı şekilde sayfalarınıza ekleyebilirsiniz.
 - **Dış/External CSS:** Kullanacağınız css dosyasının linkini **head** kısmında vermelisiniz. Örnek
 - **İç/Internal CSS:** Eğer tek bir HTML sayfasının benzersiz bir stili varsa **head** kısmında **style** etiketiyle ilgili css'i tanımlayabilirsiniz. Örnek
 - **Satır içinden/Inline:** Tek bir elemente benzersiz bir stil verecekseniz style özelliğini kullanmalısınız. Örnek
- Eğer aynı elementin farklı stilleri farklı css dosyalarında tanımlanmışsa en son okunan css dosyası kullanılır. Örnek
- Eğer stil dış kaynağın linkinden önce tanımlanmışsa dış kaynaktaki stil kullanılır. Örnek
- **Öncelik Sırası:** Inline, Internal, External Örnek



CSS Temelleri



- Yorum satırları ekleyebilirsiniz. Örnek
- Renkler önceden tanımlı renk isimleriyle ya da RGB, HEX, HSL, RGBA, HSLA değerleriyle belirlenir. Örnek-1, Örnek-2
- CSS background özellikleri elementlere arka plan efektleri ekler.
 - background-color
 - background-image
 - background-repeat, no-repeat
 - background-position
 - background-attachment
- **border-style** özelliği ne tarz bir kenar göstereceğinizi belirler. Örnek
- **border-width** kenarın kalınlığını belirler. Örnek-1, Örnek-2
- **border-color** kenarın rengini belirler. Örnek
- **border-radius** yuvarlak bir kenar görünümünü sağlar. Örnek



CSS Temelleri

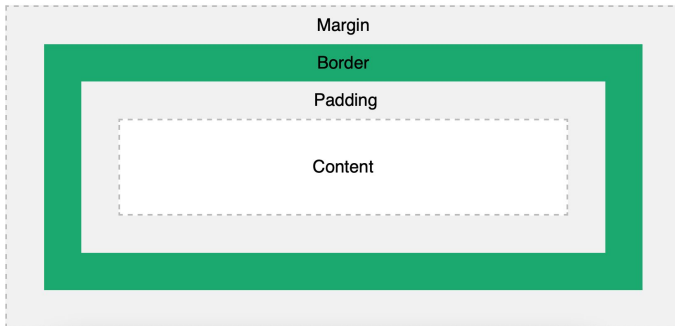


- Kenar Boşlukları/**Margins** elementler etrafında **border dışında** boşluk oluşturmayı sağlar.
- Marginler ile tüm kenarlara müdahale edebilirsiniz. Top, Bottom, Right, Left. Örnek
- Marginler şu özellikleri alabilir:
 - **auto**: tarayıcı hesaplar. Örnek
 - **length**: px, cm gibi birimlerde margin miktarları tanımlanır. Örnek
 - **%**: içeren elementin genişliğinin yüzdesi kadar ayarlama yapar.
 - **inherit**: parent elementin margin bilgilerini katılımla alır. Örnek
- **Padding**/Dolgu bir elementin içeriği etrafında **border içinde** boşluk oluşturmayı sağlar.
- Padding şu özellikleri alabilir:
 - **length**: px, cm gibi birimlerde dolgu miktarları tanımlanır. Örnek
 - **%**: içeren elementin genişliğinin yüzdesi kadar ayarlama yapar.
 - **inherit**: parent elementin padding bilgilerini katılımla alır. Örnek



CSS Temelleri

- **height** ve **width** özellikleri bir elementin boyutlarını ayarlar. Örnek
- **max-width** maksimum genişliği ayarlar. Örnek
- **CSS Box Model:** Tasarım ve yerleşimle ilgili bir kavramdır. HTML elementlerini sarmalayan bir yapıdır. Margin, Border, Padding ve içerikten/content oluşur. Örnek
 - **Content:** Metin ve resimlerin görüldüğü kısım
 - **Padding:** İçerik etrafındaki alanı temizler ve transparandır.
 - **Border:** İçerik ve padding etrafında oluşturulan kısım.
 - **Margin:** Border dışında kalan alanı temizler ve transparandır.



```
div {  
  width: 320px;  
  padding: 10px;  
  border: 5px solid gray;  
  margin: 0;  
}
```

320px (width)
+ 20px (left + right padding)
+ 10px (left + right border)
+ 0px (left + right margin)
= 350px

CSS Temelleri



- CSS metin formatlamak, font ayarlamak için bir çok özelliğe sahiptir.
 - **background-color** ile arka plan rengi ayarlanır. **color** ile metin rengi ayarlanır. Örnek
 - **text-align** ile metni yatayda hizalayabilirsiniz. Örnek
 - **text-decoration** ile altı, üstü, üzeri çizili tarzda metinler oluşturabilirsiniz. Örnek
 - **text-transform** ile büyük küçük harf ayarları yapabilirsiniz. Örnek
 - **font-family** ile metnin fontunu ayarlayabilirsiniz. Örnek
 - **font-style** ile fontun stilini belirleyebilirsiniz. Örnek
 - **font-size** ile fontun büyüklüğünü ayarlayabilirsiniz. Örnek
- Bootstrap glyphs içindeki ikonları ekleyebilirsiniz. Örnek
- Google'ın ikonlarını ekleyebilirsiniz. Örnek
- Linklere stil verebilirsiniz. Örnek



CSS Temelleri



- Listelere stil verebilirsiniz. Örnek-1, Örnek-2, Örnek-3
- Tabloların görünümüleri CSS kullanılarak daha güzel hale getirilebilir. Örnek-1, Örnek-2, Örnek-3
- CSS'te yerleşimleri kontrol etmeyi sağlayan özelliklerden biri **display** özelliğidir. Bir elementin nasıl gösterileceğini belirler. Bunlar block ya da inline olabilir.
 - Block düzeyinde elementler her zaman yeni bir satırdan başlar ve sayfanın tüm genişliğini kullanırlar. div, h1-h6, p, form, header, footer, section
 - Inline düzeyinde elementler yeni bir satırdan başlamazlar ve gerektiği kadar genişliğe sahiptirler. span, a, img
- **display:none;** genel olarak Javascript ile elementlere müdahale etmeden onları gizlemeyi sağlar. Örnek
- Position özelliği elementlerin ne tür pozisyon alacaklarını belirler.
- 5 çeşit pozisyon metodu vardır:
 - **static, relative, fixed, absolute or sticky**



CSS Temelleri

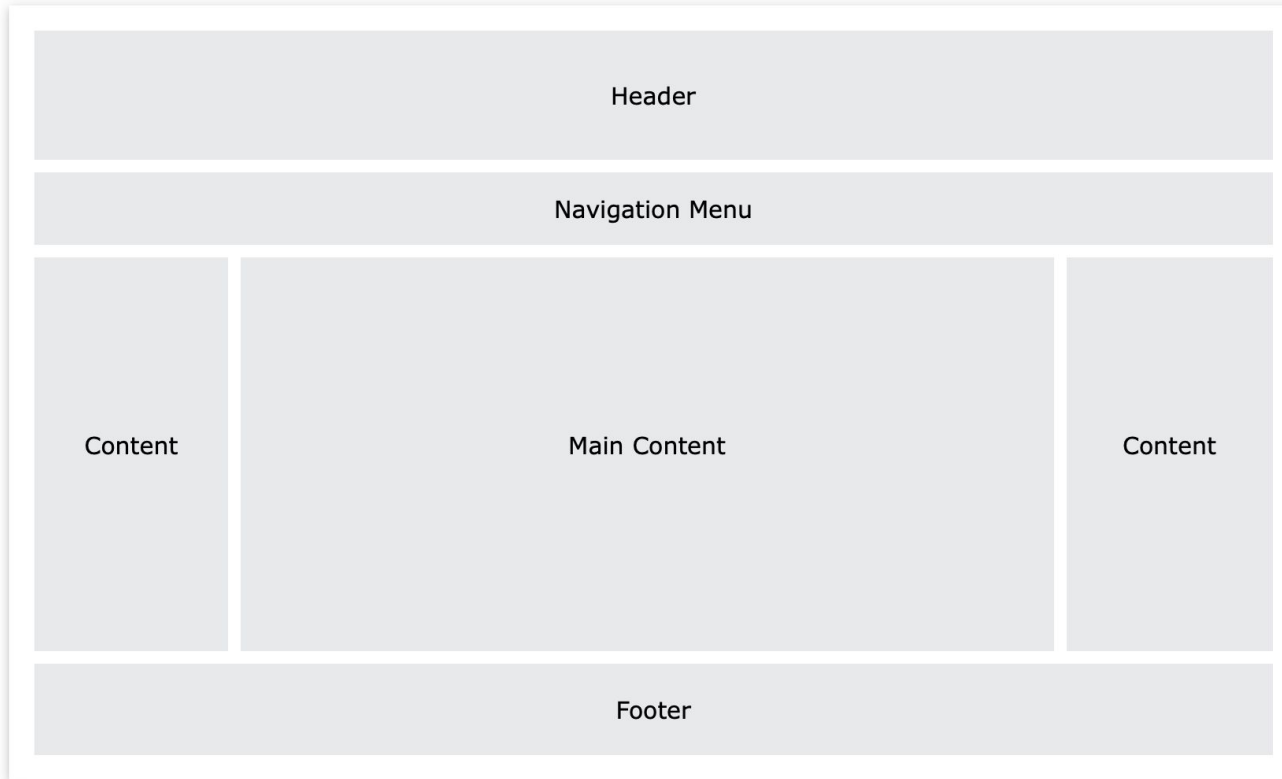


- Pozisyon metotları:
 - **static:** HTML elementleri statik olarak pozisyonlanır. top, left, right, bottom gibi özelliklerden etkilenmezler. Sayfanın normal akışına göre hizalanır. Örnek
 - **relative:** Normal pozisyonuna göreceli pozisyonlanır. Top, right vb. özellikler elementin normal pozisyonundan uzaklaşmasını sağlar. Örnek
 - **fixed:** viewport'a göre pozisyonlama sağlar. Sayfa aşağı kaydırılsa bile element yerinde kalır. Örnek
 - **absolute:** viewport yerine en yakın konumlandırılmış ata elemente göre konumlanma sağlar. Atası yoksa body kısmına göre konumlanır ve sayfa kaydırma ile birlikte hareket eder. Örnek
 - **sticky:** Kullanıcının kaydırma pozisyonuna göre yerleşim sağlar. Örnek
- Overflow özelliği içerik ilgili alana sığamayacak kadar çok büyükse içeriğin nasıl yerleşeceğini belirler. Örnek-1, Örnek-2, Örnek-3, Örnek-4, Örnek-5
- Float özelliği bir elementin nasıl yaslanması gerektiğini belirler. Örnek
- Div gibi bir block elementini yatayda ortalamak için **margin: auto;** özelliğini kullanabilirsiniz. Örnek
- Sadece bir element içindeki metni ortalamak için text-align: center; kullanabilirsiniz. Örnek

CSS Temelleri



- Bir websitesi genelde header, menu, content, footer bölümlerine sahiptir.
- Web siteleri için kullanılabilecek tonlarca yerleşim biçimi/layout mevcuttur. En popülerleri aşağıdadır.



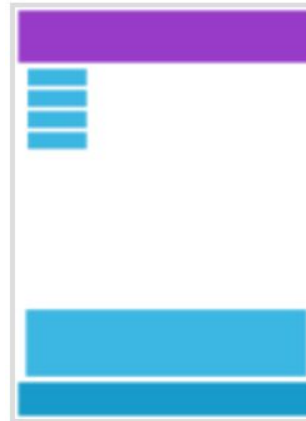
CSS Temelleri



- **Responsive/Mobil Uyumlu Tasarım:** İçeriği herhangi bir ekranda iyi görünmesi için yeniden boyutlandırmak, gizlemek, küçültmek, büyütme veya taşımak için CSS ve HTML kullandığınızda buna responsive web tasarımı denir.
- Web sayfaları birçok farklı cihaz kullanılarak görüntülenebilir: masaüstü bilgisayarlar, tabletler ve telefonlar. Web sayfanız, cihazdan bağımsız olarak iyi görünmeli ve kullanımı kolay olmalıdır.
- Web sayfaları, bilgileri daha küçük cihazlara uyacak şekilde dışarıda bırakmamalı, bunun yerine içeriğini herhangi bir cihaza uyacak şekilde uyarlamalıdır. Örnek



Masaüstü



Tablet



Telefon

CSS Temelleri



- **Viewport/Görünür Alan:** Bir web sayfasının kullanıcıya görünür alanıdır.
- Viewport cihaza göre değişir ve cep telefonunda bilgisayar ekranından daha küçüktür.
- Tabletler ve cep telefonlarından önce web sayfaları sadece bilgisayar ekranları için tasarlanıyordu ve web sayfalarının statik bir tasarıma ve sabit bir boyuta sahip olması yaygındı.
- Daha sonra tabletler ve cep telefonları kullanarak internette gezinmeye başladığımızda, sabit boyutlu web sayfaları görüntü alanına sığmayacak kadar büyüktü.
- Bunu düzeltmek için, bu cihazlardaki tarayıcılar tüm web sayfasını ekrana sığacak şekilde küçülttü.
- HTML5, web tasarımcılarının <meta> etiketi aracılığıyla görünüm üzerinde kontrolü ele geçirmesine izin veren bir yöntem sundu.



CSS Temelleri



- Tüm web sayfalarınıza aşağıdaki <meta> görünüm alanı ögesini eklemelisiniz:
 - `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- Bu kod, tarayıcıya sayfanın boyutlarını ve ölçeklendirmesini nasıl kontrol edeceğine dair talimatlar verir.
- **width=device-width** kısmı, sayfanın genişliğini cihazın ekran genişliğini takip edecek şekilde ayarlar (cihaza göre değişir).
- **initial-scale=1.0** bölümü, sayfa tarayıcı tarafından ilk yüklendiğinde başlangıç yakınlaştırma düzeyini ayarlar.
- Kullanıcılar, web sitelerini hem masaüstü hem de mobil cihazlarda dikey olarak kaydırmak için kullanılır - ancak yatay olarak değil!
- Bu nedenle, kullanıcı tüm web sayfasını görmek için yatay olarak kaydırmaya veya uzaklaştırmaya zorlanırsa, bu kötü bir kullanıcı deneyimiyle sonuçlanır.
Bu nedenle:
 - Geniş sabit uzunlukta elementler kullanmayın.
 - Ekran boyutları değişebildiği için içeriğinizin genişliği tek bir viewport'a göre ayarlanmamalı.



CSS Temelleri



- Her zaman ilk olarak mobil uyumlu tasarım yapın. Bu, sayfanın daha küçük cihazlarda daha hızlı görüntülenmesini sağlar.
 - **Önce Mobil/Mobile First:** Masaüstü veya başka bir cihaz için tasarım yapmadan önce mobil için tasarım yapmak.
- Bu, CSS'mizde bazı değişiklikler yapmamız gerektiği anlamına gelir.
- Genişlik 768px'den küçük olduğunda stilleri değiştirmek yerine, genişlik 768px'den büyük olduğunda tasarımı değiştirmeliyiz.
- Bu, tasarımımızı **Mobile First** yapacaktır.
- Resimlerin de mobil uyumlu görünmesini sağlayabiliriz.
- **width** özelliği bir yüzdeye (%) ve **height** özelliği «auto» olarak ayarlanmışsa, görüntü responsive olacak ve yukarı ve aşağı ölçeklenecektir. Örnek



Javascript Temelleri



- HTML'de javascript `<script></script>` etiketleri içine yazılır.
- HTML'e istediğiniz kadar script ekleyebilirsiniz.
- Javascript HTML'de genelde **head** kısmına yazılır. Örnek
- Dış kaynaktan bir dosya içinden de çağrılabilir. Bu şekilde HTML ile scriptlerinizi ayırmış olursunuz. Okunabilirlik artar ve bakım maliyeti düşer. Önbelleğe alınan javascript daha hızlı işletilir.
- Dış kaynaklardaki javascripti projenize 3 farklı şekilde dahil edebilirsiniz.
 - Web adresi: `<script src="https://www.asimsinan.com/js/myscript.js"></script>`
 - Dosya yolu: `<script src="/js/myscript.js"></script>`
 - Herhangi bir yol olmadan: `<script src="myscript.js"></script>`



Javascript Temelleri



- Veri farklı şekillerde gösterilebilir.
 - **innerHTML** ile bir HTML elementinin içine: Örnek
 - **document.write** ile HTML çıktısı olarak (Sadece test amaçlı kullanılmalıdır): Örnek
 - Alert kutusu ile: **window.alert()**: Örnek
 - console.log() ile tarayıcı konsoluna (debug amaçlı): Örnek
- Javascript ifadeleri ; ile ayrılır. Satır sonuna yine ; işareti eklenir.
- Boşlukları dikkate almaz.
 - let ogrenci = "Sinan"; let ogrenci="Sinan";
- Javascript ifadeleri {...} ile kod blokları halinde yazılabilir.
 - function topla(){.....}
- Anahtar kelimeler: var, let, const, if, switch, for, function, return



Javascript Temelleri



- var, let, const değişken tanımlamak için kullanılır. var eskidir, let yeni. İkisi de aynı işi yapar.
 - var sayi=1; let x,y; const pi=3.14
- **const** ile tanımladığınız nesne ya da diziler değişkenler sabit değer değil sabit bir referans alır. Dizideki elemanları, nesnenin özelliklerini değiştirebilirsiniz ancak yeni bir dizi, nesne atayamazsınız.
- Javascript büyük küçük harf duyarlıdır.
 - Ad-soyad, adSoyad farklı değişkenlerdir.
- Geliştiriciler değişken, fonksiyon tanımlarında deve notasyonu (camelCase) kullanırlar.
- String değişkenler tek ya da çift tırnak içinde yazılabilir.
 - let ad='Sinan'; let soyad = "Yüksel";
- Değer atanmadan tanımlanan değişkenler «**undefined**» değerindedir.



Javascript Temelleri



- Aritmetik operatörler: `+`, `-`, `*`, `**`, `/`, `%`, `++`, `--`
- Atama operatörleri: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`
- Karşılaştırma operatörleri: `==`, `!=`, `>`, `<`, `<=`, `>=`, `?`
- Mantıksal operatörler: `&&` (and), `||` (or), `!` (not)
- Tip operatörleri: `typeof`, `instanceof`
- Bit düzeyinde operatörler: `&` (AND), `|` (OR), `~` (NOT), `^` (XOR), `<<` (left shift), `>>` (right shift),
- Veri türleri: sayı, string, nesne (object), dizi
- Veri tipleri dinamiktir.
 - `let x; x=5; x="Sinan"`
- Dizi tanımı: `const arabalar = ["Opel", "Mercedes", "BMW"];`
- Nesne tanımı:
 - `const kullanıcı = {ad:"Sinan", soyad:"Yüksel", tcNo: 1234}`



Javascript Temelleri



- Fonksiyon tanımı:

```
function fonksiyonAdi(parametre1, parametre2, parametre3) {  
    // çalıştırılacak kod  
}
```

```
let x = topla(4, 3);    // Fonksiyon çağrılır ve sonuç x değişkenine aktarılır
```

```
function topla(a, b) {  
    return a + b;        //İki sayının toplamını döndüren fonksiyon  
}
```

- Fonksiyonların içinde tanımlanan değişkenler yerel değişkenlerdir. Fonksiyon çalışmaya başladığında aktif olurlar, bittiğinde silinirler. Dolayısıyla farklı fonksiyonlarda aynı isimle değişken oluşturabilirsiniz.
- Nesne tanımlama, değişkenlere, metotlara erişim Java dilinde olduğu gibidir ancak tek fark nesne {} içinde tanımlanır. (JSON ifadesi gibi)



Javascript Temelleri



- HTML'de events/olaylar HTML elementlerinin başına gelen şeylerdir.
- HTML içinde Javascript kullandığınızda Javascript bu olaylara tepki verebilir, bir fonksiyonu aktif edebilir. Örnek
 - HTML sayfası yüklendiğinde
 - Bir input içeriği değiştiğinde
 - Bir düğmeye tıklandığında
- Sık kullanılan HTML olayları:
 - onchange: Bir HTML elementi değiştiğinde
 - onclick: Kullanıcı bir HTML elementine tıkladığında
 - onmouseover: Kullanıcı fareyi bir HTML elementinin üzerine getirdiğinde
 - onmouseout: Kullanıcı fareyi bir HTML elementinin üzerinden çektiğinde
 - onkeydown: Kullanıcı klavyeden bir tuşa bastığında
 - onload: Tarayıcı sayfayı yüklemeyi bitirdiğinde



Javascript Temelleri



- **If:**

```
if (koşul1) {  
    //koşul1 sağlandığında işletilecek kod  
} else if (koşul2) {  
    //koşul2 sağlandığında işletilecek kod  
} else {  
    //koşul1 ve koşul2 sağlanmadığında işletilecek kod  
}
```

- **Switch:**

```
switch(ifade) {  
    case x:  
        //kod bloğu  
        break;  
    case y:  
        //kod bloğu  
        break;  
    default:  
        // kod bloğu  
}
```



Javascript Temelleri



- **for loop:**

```
for (koşul 1;koşul 2;koşul 3) {  
    // işletilecek kod bloğu  
}
```

- Örnek

- **for in loop:**

```
for (anahtar in nesne) {  
    // işletilecek kod bloğu  
}
```

- Örnek

- Array.forEach()



Javascript Temelleri



- **Arrow fonksiyonu:** fonksiyonları daha kısa bir formatta yazmayı sağlar.

- let carpma = (a, b) => a * b;

```
hello = function() {  
    return "Hello World!";  
}
```

```
hello = () => {  
    return "Hello World!";  
}
```

- **Sınıf tanımlama:**

```
class ClassName {  
    constructor() { ... }  
    method_1() { ... }  
    method_2() { ... }  
    method_3() { ... }  
}
```

- Örnek-1, Örnek-2, Örnek-3



Javascript Temelleri



- Bazen fonksiyonların çalışma biçimini kontrol etmek isteyebilirsiniz. Mesela önce hesaplamaların yapılıp, bitmesini daha sonra ekranda gösterilmesi gibi.
- Javascript bu problemi çözmek için **callback** fonksiyonu oluşturmaya izin verir. Amaç bir fonksiyonu başka bir fonksiyona parametre olarak geçirmek ve ana fonksiyonun işi bittiğinde bu callback fonksiyonu çağırmak. Örnek
- Gerçek hayatta callback asenkron fonksiyonlar için kullanılır. setTimeout gibi.
- Bir dosyanın içeriği yüklenmeden başka bir iş yapmak istemiyorsanız yine callback kullanabilirsiniz. Örnek
- Bazen üretim yapan bir metodun çalışması uzun sürebilir ve tüketim metodunun üretimi yapan metodun sonuçlarını beklemesi gerekebilir. **Promise** bu problemi çözmek için kullanılır.
- Promise 3 durumdan birinde olabilir: pending, fulfilled, rejected. Duruma göre oluşabilecek sonuçlar vardır. Örnek
 - Pending: undefined
 - Fulfilled: dönüş değeri
 - Rejected: hata nesnesi
- Promise olmayan metodu **async** ile promise'e dönüştürebilirsiniz. Örnek
- **await** ile bir fonksiyonun bir promise'i beklemesini sağlayabilirsiniz. await sadece async bir fonksiyonun içinde kullanılabilir. Örnek



Javascript Temelleri



- Javascript HTML DOM (Document Object Model) kullanarak HTML dokümanının tüm elementlerini değiştirebilir.
- Tüm HTML elementlerini, tüm HTML özelliklerini, tüm CSS stillerini, değiştirebilir, kaldırabilir, yeni element, özellik ya da stil ekleyebilir.
- HTML DOM standart bir nesne modelidir ve HTML için bir programlama arayüzüdür.
 - HTML elementlerini nesne olarak tanımlar.
 - HTML özelliklerinin tümünü tanımlar.
 - HTML özelliklerine erişmek için gerekli metotları tanımlar.
 - HTML elementlerinin olaylarını/events tanımlar.
- **document.getElementById(id)** Bir elemente id'sine göre erişebilir. **innerHTML** ile içeriğine erişebilir. Örnek
- document.getElementsByTagName(name) verilen element ismine uyan tüm elementleri döndürür.
- document.getElementsByClassName(name) verilen sınıf ismine sahip tüm elementleri döndürür.



Javascript Temelleri



- `element.attribute` ile bir elementin özelliğinin aldığı değeri değiştirebilir.
- `element.style.property` ile stil özelliğini değiştirebilir.
- `element.setAttribute(attribute, value)` ile bir elemente ait özelliğin değerini değiştirebilir.
- `document.createElement(element)` ile yeni bir element oluşturabilir.
- `document.removeChild(element)` ile bir elementi kaldırabilir.
- `document.appendChild(element)` ile bir element ekleyebilir.
- `document.replaceChild(new, old)` ile eski bir elementi yenisiyle değiştirebilir.
- `document.getElementById(id).onclick = function(){..}` ile bir elemente fonksiyon bağlayabilir. Örnek
- Fare hareketlerine fonksiyon bağlayabilir. Örnek
- HTML formlarını doğrulama işlemi yapabilir. Örnek



Javascript Temelleri

- HTML dokümanındaki her şey bir düğümdür/node ve Javascript tüm düğümlere erişebilir.
- html kök düğümdür. head hmtl'in ilk çocuğu, body ikinci çocuğudur.
- head'in title isminde bir çocuğu vardır. title'in «My title» isminde bir çocuğu vardır.
- body'nin a ve h1 adında iki çocuğu vardır.

